Payment Service Integration Project

1. Overview

Marketly is an online store that is already running. It has users, products, a cart, orders and stock, but it cannot take payments. Your task is to build the **Payment Service**: a separate backend that Marketly calls to start, check and refund payments. Paystack in test mode is the payment gateway, and customers can pay by card or bank transfer.

This mirrors how payments are added to real systems. Marketly owns orders and stock; your service owns everything to do with money and is the only component that talks to Paystack. Customers pay on Paystack's hosted checkout page, so card details never reach your servers. Paystack reports outcomes to your service through **webhooks**, and your service reports them to Marketly through **signed callbacks**.

You build only the Payment Service: no frontend, user accounts or product catalog. Complete the Environment Setup Guide before starting. Every submission will be tested against a common boilerplate application provided by the instructor, so follow the specification in Section 3 exactly. Use any language and framework, with a database that persists across restarts.

2. Payment Flow

1. Marketly creates an order and calls POST /payments with the order ID, amount and an idempotency key.
2. Your service saves a pending payment, initialises a Paystack transaction using the payment ID as its reference, and returns the checkoutUrl.
3. The customer pays on Paystack by card, or leaves to make a bank transfer that may arrive minutes later.
4. Paystack sends charge.success to your webhook. You verify the signature, confirm the transaction with Paystack's verify endpoint, and mark the payment succeeded.
5. Your service sends a signed callback to Marketly and retries until Marketly accepts it. Expiry and refunds follow the same pattern.

Suggested structure

```
src/
  config/       load and validate environment variables
  db/           schema and migrations
  payments/     routes, state machine, idempotency
  refunds/      refund logic
  webhooks/     Paystack signature check, processed-event log
  callbacks/    outbox table, signing, retry worker
  jobs/         expiry and reconciliation worker
  providers/    Paystack hidden behind a PaymentProvider interface
```

3. Specification

3.1 API

All endpoints except /health and /webhooks/paystack require Authorization: Bearer <MERCHANT_API_KEY> and return 401 without it. Amounts are integers in kobo and the only currency is NGN.

Endpoint	Behaviour
GET /health	200 { "status": "ok" }
POST /payments	Requires Idempotency-Key. Body: orderId, amount, currency, customerEmail, returnUrl. Returns 201 { paymentId, orderId, status, amount, currency, checkoutUrl }. 400 for invalid input or a missing key; 409 payment_exists if the order already has a pending or succeeded payment; 422 if a key is reused with a different body. A repeated key with the same body returns the original response with the header Idempotent-Replayed: true.
GET /payments/:id	Returns the payment with status, channel and amountRefunded; 404 if unknown.
POST /payments/:id/refunds	Requires Idempotency-Key. Optional amount (defaults to the remaining balance). Returns 202 { refundId, amount, status: "pending" }, or 409 not_refundable.

Endpoint	Behaviour
POST /webhooks/paystack	Receives Paystack events.
POST /admin/reset	Only when TEST_MODE=true. Clears all data and returns 204.

3.2 States

Payment: pending → succeeded or expired; expired → succeeded (a verified late transfer only); succeeded → partially_refunded → refunded. **Refund:** pending → processed or failed. Any other transition must be rejected in code.

3.3 Callbacks to Marketly

Send payment.succeeded, payment.expired, payment.refunded and payment.refund_failed as a POST to CALLBACK_URL with the body { id, type, createdAt, data: { paymentId, orderId, status, amount, amountRefunded, currency, late } }. Sign each callback with HMAC-SHA256 over timestamp.body using CALLBACK_SECRET, sent as X-Marketly-Signature: t=<unix>,v1=<hex>.

4. Requirements

A. Idempotency

- Store every idempotency key with its request hash and response under a unique database constraint. Repeats return the saved response; never a second payment or Paystack transaction.
- Use your payment ID as the Paystack reference, created and saved before calling Paystack. Paystack rejects duplicate references, which protects you if you retry after a timeout.
- Apply the same rules to refunds.

B. Concurrency

A **race condition** occurs when two operations run at the same time and interfere. Each case below must be safe under parallel requests, using database guarantees (unique constraints, row locks or conditional updates) rather than read-then-write checks in application code.

- Ten parallel requests with the same idempotency key create exactly one payment.
- Parallel requests for the same orderId with different keys create at most one active payment.
- A charge.success webhook and the expiry job processing the same payment at the same moment leave it in one correct state.
- Concurrent refund requests can never reserve more than the amount paid. Pending refunds count against the balance; failed refunds release it.

C. Paystack webhooks

- Verify x-paystack-signature: an HMAC-SHA512 of the **raw** request body using your secret key. Invalid signatures return 400 and change nothing.
- Paystack events have no event ID, so build a deduplication key such as event:data.id and record it in the same transaction as the state change. Duplicates return 200 and do nothing.
- Before marking a payment succeeded, call GET /transaction/verify/:reference and confirm the status, amount and currency match your record.
- Handle charge.success, refund.processed and refund.failed. Acknowledge other events with 200. Respond quickly and do slow work afterwards.

D. Reliable callbacks

Write each callback to an outbox table in the same transaction as the state change, then deliver it from a background worker. Retry any non-2xx response or timeout with exponential backoff (1s, 2s, 4s and so on, at least 6 attempts), keeping the same id so Marketly can discard duplicates. Undelivered callbacks must survive a restart.

E. Expiry, bank transfers and reconciliation

- Paystack sends no event when a customer abandons checkout. A background job must check payments still pending after PAYMENT_TIMEOUT_SECONDS with the verify

endpoint: mark them succeeded if Paystack shows success (a missed webhook), otherwise expired.

- Bank transfers can arrive after a payment has expired. A verified late charge.success moves the payment to succeeded and sends a callback with late: true; Marketly decides what to do with the order.
- The same job doubles as reconciliation: your database and Paystack must agree even if webhooks were lost while your service was down.

F. Refunds and failure handling

- Support full and partial refunds through Paystack's refund API. Final status comes only from refund.processed or refund.failed.
- Retry network errors and Paystack 5xx responses with backoff. Never retry declines or validation errors.
- If a call to Paystack times out, you cannot assume it failed. Be ready to explain how your service recovers.

5. Testing and Submission

Your service will be run against the instructor's boilerplate application acting as Marketly. Fake Paystack webhooks will be signed with your test secret key to check replays and ordering. Each check is pass or fail.

Area	What is checked	Points
Basics	Health, 401 without the API key, card payment, declined card then retry, bank transfer	20
Idempotency	Sequential and parallel repeats, 400, 409 and 422 cases	15
Concurrency	Parallel creates for one order, webhook racing expiry, concurrent refunds	15
Webhooks	Bad signatures rejected, duplicates ignored, amount verified	15
Callbacks	Valid signature and shape, retried with the same ID while Marketly is failing	15

Area	What is checked	Points
Expiry	Abandoned payments expire; a late transfer is reported with late: true	10
Restart	Data, idempotency keys and undelivered callbacks survive a restart	10

Submission. Submit to the github organization repo in this format: payment_service_1...10